

1. Qué pretende enseñar el ejemplo

El objetivo pedagógico es muy bueno: tener dos endpoints:

/gfg → requiere autenticación

/gfg/welcome → acceso público

El original usa:

```
.antMatchers("/gfg").authenticated()
```

```
.antMatchers("/gfg/welcome").permitAll()
```

y un usuario en memoria:

Usuario: gfg

Contraseña: gfg123

2. Crear el proyecto moderno

proyecto **Spring Boot** con:

- Maven
- Java 21
- Spring Web
- Spring Security

Una estructura sencilla sería:

```
springsecurity/
|
├─ src/main/java/com/ejemplo/security/
|   |
|   └─ SecurityApplication.java
|   |
|   └─ controller/
|       └─ GfgController.java
|   |
|   └─ config/
|       └─ SecurityConfig.java
|
├─ src/main/resources/
|   └─ application.properties
|
└─ pom.xml
```

3. pom.xml

No necesitamos esto:

```
<dependency>

    <groupId>javax.servlet</groupId>

    <artifactId>javax.servlet-api</artifactId>

    <version>4.0.1</version>

    <scope>provided</scope>

</dependency>
```

Tampoco necesitamos declarar individualmente:

spring-webmvc

spring-security-config

spring-security-web

Spring Boot gestiona esas dependencias mediante los starters.

Podemos utilizar:

```
<dependencies>
```

```
    <dependency>

        <groupId>org.springframework.boot</groupId>

        <artifactId>spring-boot-starter-web</artifactId>

    </dependency>
```

```
    <dependency>

        <groupId>org.springframework.boot</groupId>

        <artifactId>spring-boot-starter-security</artifactId>
```

```
</dependency>
```

```
<dependency>
```

```
    <groupId>org.springframework.boot</groupId>
```

```
    <artifactId>spring-boot-starter-test</artifactId>
```

```
    <scope>test</scope>
```

```
</dependency>
```

```
</dependencies>
```

4. Clase principal

```
package com.ejemplo.security;
```

```
import org.springframework.boot.SpringApplication;
```

```
import org.springframework.boot.autoconfigure.SpringBootApplication;
```

```
@SpringBootApplication
```

```
public class SecurityApplication {
```

```
    public static void main(String[] args) {
```

```
        SpringApplication.run(SecurityApplication.class, args);
```

```
    }
```

```
}
```

Aquí tampoco necesitamos crear manualmente el DispatcherServlet.

Spring Boot se encarga de ello.

5. Crear el controlador

```
package com.ejemplo.security.controller;
```

```
import org.springframework.web.bind.annotation.GetMapping;  
import org.springframework.web.bind.annotation.RestController;
```

```
@RestController
```

```
public class GfgController {
```

```
    @GetMapping("/gfg")
```

```
    public String privado() {
```

```
        return "Bienvenido al endpoint protegido";
```

```
    }
```

```
    @GetMapping("/gfg/welcome")
```

```
    public String publico() {
```

```
        return "Bienvenido al endpoint público";
```

```
    }
```

```
}
```

Tenemos entonces:

GET /gfg

GET /gfg/welcome

6. Configurar Spring Security

Creamos:

config/SecurityConfig.java

con:

```
package com.ejemplo.security.config;
```

```
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.provisioning.InMemoryUserDetailsManager;
import org.springframework.security.web.SecurityFilterChain;
```

@Configuration

```
public class SecurityConfig {
```

@Bean

```
public SecurityFilterChain securityFilterChain(HttpSecurity http)
```

```
    throws Exception {
```

```
    http
```

```
        .authorizeHttpRequests(auth -> auth
```

```
            // Endpoint público
```

```
            .requestMatchers("/gfg/welcome").permitAll()
```

```
            // Endpoint protegido
```

```

        .requestMatchers("/gfg").authenticated()

        // Cualquier otro endpoint
        .anyRequest().authenticated()
    )

    // Autenticación mediante formulario
    .formLogin(form -> form
        .permitAll()
    )

    // También permite HTTP Basic
    .httpBasic(basic -> {});

    return http.build();
}

```

@Bean

```

public UserDetailsService userDetailsService(
    PasswordEncoder passwordEncoder) {

    UserDetails usuario = User.builder()
        .username("gfg")
        .password(passwordEncoder.encode("gfg123"))
        .roles("USER")
        .build();
}

```

```
        return new InMemoryUserDetailsManager(usuario);  
    }  
}
```

@Bean

```
public PasswordEncoder passwordEncoder() {  
    return new BCryptPasswordEncoder();  
}  
}
```

7. antMatchers() → requestMatchers()

SPRING SECURITY 5 SPRING SECURITY 6

antMatchers() → requestMatchers()

authorizeRequests() → authorizeHttpRequests()

WebSecurityConfigurerAdapter

→ SecurityFilterChain

Por ejemplo:

Antiguo

```
.antMatchers("/gfg").authenticated()  
.antMatchers("/gfg/welcome").permitAll()
```

Actual

```
.requestMatchers("/gfg/welcome").permitAll()  
.requestMatchers("/gfg").authenticated()
```

8. Primera prueba

Ejecutamos:

```
mvn spring-boot:run
```

Ahora abrimos:

```
http://localhost:8080/gfg/welcome
```

Debe aparecer directamente:

Bienvenido al endpoint público

No debe pedir usuario ni contraseña.

Esto demuestra:

```
.requestMatchers("/gfg/welcome").permitAll()
```

9. Segunda prueba

Ahora:

```
http://localhost:8080/gfg
```

Spring Security debe solicitar autenticación.

Utilizamos:

Usuario: gfg

Contraseña: gfg123

Después de autenticarnos veremos:

Bienvenido al endpoint protegido

Esto demuestra:

```
.requestMatchers("/gfg").authenticated()
```